



Sistema de Gerenciamento de Tráfego - 2026.1

Centro de Ciência e Tecnologia - CCT

TRABALHO DE SISTEMAS OPERACIONAIS

Aisha Tomaz | Elilúcio Teixeira | Icaro Cavalcante | Samuel Jackson

SISTEMAS OPERACIONAIS

Concorrência, Sincronização e Deadlocks

Relatório do Projeto Prático

Simulador de Tráfego Urbano em C

Entrega: 10/07/2026

Sumário

1. Introdução e Objetivo
2. Equipe e Responsabilidades
3. Representação do Mapa
4. Organização das Threads
5. Mecanismos de Sincronização
6. Ausência de Espera Ocupada
7. Tratamento da Ambulância
8. Estratégia contra Deadlocks
9. Visualização
10. Decisões de Implementação e Trade-offs
11. Aprendizados da Equipe
12. Compilação e Execução
13. Checklist de Atendimento aos Requisitos
14. Conclusão
15. Repositório

1. Introdução e Objetivo

Este relatório documenta o desenvolvimento do Simulador de Tráfego Urbano em C, projeto prático da disciplina de Sistemas Operacionais. O trabalho consiste em simular uma malha viária urbana onde veículos — representados por threads concorrentes — disputam o acesso a células da pista, cruzamentos e semáforos, aplicando na prática conceitos de exclusão mútua, espera bloqueante, variáveis de condição e prevenção de deadlocks.

O texto a seguir explica as principais decisões de implementação adotadas pela equipe: a representação do mapa, a organização das threads, os mecanismos de sincronização utilizados, a ausência de espera ocupada (busy-wait), a estratégia contra deadlocks e o tratamento de prioridade da ambulância. Trechos de código reais do repositório são citados ao longo do texto para embasar as explicações.

2. Equipe e Responsabilidades

O projeto foi dividido em quatro módulos, cada um sob responsabilidade de um integrante, seguindo a organização definida no início do trabalho:

Integrante	Área de responsabilidade	Arquivos principais
Aisha	Mapa e Renderização	include/map.h, src/map.c, include/render.h, src/render.c
Elilúcio	Veículos e Threads	include/vehicle.h, src/vehicle.c
Jackson	Sincronização	include/sync.h, src/sync.c
Icaro	Relógio, Main e Integração	include/clock.h, src/clock.c, src/main.c, Makefile

Essa divisão seguiu uma ordem de dependências: o módulo de mapa (Aisha) não depende de nenhum outro e foi o primeiro a ser publicado; sincronização (Jackson) depende apenas do mapa; veículos (Elilúcio) depende do mapa e da sincronização; e a integração final (Icaro) depende de todos os módulos anteriores.

[DEPENDÊNCIAS PREVISTAS] Os arquivos que geriam o mapa foram desenvolvidos com prioridade, seguido da implementação do módulo de sincronização, o que destravou o desenvolvimento das lógicas de veículos e clocks. A equipe, ao final do desenvolvimento das features, implementou em conjunto os arquivos de Makefile e main.c.

3. Representação do Mapa

A malha viária é representada por uma matriz estática `Cell grid[MAX_ROWS][MAX_COLS]` (30×30), dentro da struct `Map`. Cada célula guarda seu tipo (`CellType`), o identificador do veículo que a ocupa no momento (`occupant_id`, -1 quando livre) e o símbolo usado para desenhá-la na tela (`occupant_symbol`).

```
typedef struct {
    CellType type;
    int occupant_id;
    char occupant_symbol;
} Cell;

typedef struct {
    Cell grid[MAX_ROWS][MAX_COLS];
    int rows;
    int cols;
} Map;
```

O mapa é carregado a partir de um arquivo de texto (assets/map.txt) por `map_load()`, que lê o arquivo linha a linha, valida que todas as linhas têm o mesmo número de colunas (mapa retangular) e converte cada caractere em um `CellType` através de `char_to_type()`. A legenda utilizada é a seguinte:

Caractere	Significado
#	Parede / não é via
+	Cruzamento (semáforo)
^	Via de mão única, sentido Norte (linha diminui)
v	Via de mão única, sentido Sul (linha aumenta)
>	Via de mão única, sentido Leste (coluna aumenta)
<	Via de mão única, sentido Oeste (coluna diminui)

```
4
5  /* -----
6   * Legenda do assets/map.txt:
7   *   '#' parede / não é via
8   *   '+' cruzamento (semáforo)
9   *   '^' via de mão única sentido norte (linha diminui)
10  *   'v' via de mão única sentido sul (linha aumenta)
11  *   '>' via de mão única sentido leste (coluna aumenta)
12  *   '<' via de mão única sentido oeste (coluna diminui)
13  *
14  *
15  * OBS: a mão dupla são duas faixas de mão única em sentidos opostos (olhar map.txt
16  * para melhor entendimento). Não existe compartilhamento de célula, pois tem dois
17  * sentidos, ou seja, cada faixa é seu próprio mutex.
18  *
19  * (09/07/26)
20  * OBS²: Não foi feita nenhuma mudança em CellType e nem em "map_isvalid_move".
21  * ----- */
```

Figura — Comentário de cabeçalho em `src/map.c` explicando a legenda de `assets/map.txt`.

Vias de mão dupla não são um tipo de célula à parte: elas são modeladas como duas faixas de mão única em sentidos opostos, lado a lado (por exemplo, uma coluna '^' ao lado de uma coluna 'v'). Isso significa que não existe compartilhamento de célula entre sentidos opostos — cada faixa tem seu próprio mutex, o que simplifica a exclusão mútua.

Os cruzamentos (célula '+') são os únicos pontos onde um veículo pode mudar de direção; `map_is_valid_move()` permite qualquer direção nessas células, enquanto nas vias de mão única só é permitido o movimento no sentido indicado. O mapa atual (`assets/map.txt`) tem 9 cruzamentos, atendendo ao mínimo de 8 exigido no enunciado.

```

1      #####
2      #++>>>>+>>>>++#
3      #++<<<<+<<<<++#
4      #^v####^####^v#
5      #^v####^####^v#
6      #^v####^####^v#
7      #^v####^####^v#
8      #++>>>>+>>>>++#
9      #++<<<<+<<<<++#
10     #^v####^####^v#
11     #^v####^####^v#
12     #^v####^####^v#
13     #^v####^####^v#
14     #++>>>>+>>>>++#
15     #++<<<<+<<<<++#
16     #####

```

Figura — Layout final do mapa (`assets/map.txt`) visto no histórico de commits do repositório.

4. Organização das Threads

A simulação lança quatro categorias de thread a partir de main.c, todas coordenadas pelo relógio global:

- Thread do relógio (clock_thread): uma única thread que avança o tick global a cada 100 ms e acorda todas as demais threads via broadcast;
- Threads de semáforo (traffic_signal_loop): uma por cruzamento — o mapa atual gera 9 threads desse tipo — cada uma controlando o ciclo de abertura/fechamento das quatro direções do seu cruzamento;
- Threads de veículo (vehicle_thread_entry → vehicle_run): uma por carro/ambulância. main.c lança NUM_TOTAL_VEHICLES (definido a partir de MAX_VEHICLE, hoje 20), reservando 1 delas para a ambulância (TYPE_AMBULANCE) e as demais como carros (TYPE_CAR), alternando entre velocidades lenta, média e rápida.

Antes de lançar cada veículo, main.c usa collect_spawn_points() para localizar células de mão única no mapa e cell_try_occupy() para garantir que dois veículos não nasçam na mesma célula. Ao final da simulação, todas as threads são encerradas com pthread_join(), garantindo que nenhum recurso seja liberado (sync_destroy(), clock_destroy(), map_free()) enquanto ainda há thread em execução.

[DECISÕES DO MAPA] Apesar de repositórios semelhantes, utilizados como “benchmark”, esboçarem um sistema de tráfego aberto, onde as Threads seriam encerradas conforme os veículos saíssem da malha do mapa, a especificação do trabalho sugere que os veículos não devem deixar o grid. Logo, foi implementado um mapa “fechado”, completamente cercado de paredes, indicadas pelo carácter “#” e as Threads são encerradas após uma contagem de tempo.

5. Mecanismos de Sincronização

O módulo sync.c centraliza toda a sincronização da simulação em uma estrutura global única, SyncContext g_sync, que contém um mutex por célula do mapa e um TrafficSignal (mutex + variável de condição) por cruzamento:

```
typedef struct {
    pthread_mutex_t cell_mutex[MAX_ROWS][MAX_COLS];
    TrafficSignal signals[N_INTERSECTIONS];
    int signal_index[MAX_ROWS][MAX_COLS];
    int num_signals;
} SyncContext;
```

A tabela abaixo resume onde cada mecanismo pedido no enunciado aparece no código:

Mecanismo	Onde aparece no código	Para que serve
Mutex	g_sync.cell_mutex[row][col] (um mutex por célula da matriz)	Protege occupant_id/occupant_symbol de cada célula em cell_try_occupy() e cell_release().

Mecanismo	Onde aparece no código	Para que serve
Mutex + Cond. de condição	<code>ClockState.mutex / .cond</code> (relógio global)	<code>clock_wait_tick()</code> dorme em <code>pthread_cond_wait()</code> até o próximo tick; <code>clock_thread()</code> dá <code>pthread_cond_broadcast()</code> a cada 100 ms.
Mutex + Cond. de condição	<code>TrafficSignal.mutex / .cond</code> (um por cruzamento)	<code>traffic_wait_green()</code> bloqueia o veículo em <code>pthread_cond_wait()</code> enquanto o sinal da sua direção estiver vermelho; <code>traffic_signal_loop()</code> faz broadcast a cada troca de fase ou pedido de prioridade.
Exclusão mútua não bloqueante ("tenta e desiste")	<code>cell_try_occupy()</code>	Em vez de bloquear esperando a célula ficar livre, o veículo tenta uma vez; se ocupada, simplesmente não avança neste tick e tenta de novo no próximo — evita reter um mutex enquanto espera outro recurso.
Thread	<code>clock_thread</code> , <code>traffic_signal_loop</code> (uma por cruzamento), <code>vehicle_thread_entry</code> (uma por veículo)	Entidades concorrentes da simulação.

5.1 Exclusão mútua das células

O `cell_try_occupy()` tranca o mutex da célula, verifica se `occupant_id` já está ocupado e, se estiver livre, grava o id do veículo; `cell_release()` faz o processo inverso. Toda leitura e escrita de `occupant_id` passa por essas duas funções, o que evita que dois veículos ocupem a mesma célula simultaneamente.

5.2 Relógio global

O `clock_wait_tick()` bloqueia a thread chamadora em `pthread_cond_wait()` até que o tick mude; `clock_thread()` incrementa o tick a cada 100 ms (`usleep`) e chama `pthread_cond_broadcast()` para acordar todas as threads esperando. Isso garante que o avanço do tempo da simulação seja centralizado e visível para todos os veículos ao mesmo tempo.

5.3 Semáforos de cruzamento

Cada cruzamento tem seu próprio `TrafficSignal`, com um estado (`SIGNAL_RED` / `SIGNAL_GREEN`) por direção. `traffic_wait_green()` bloqueia o veículo em `pthread_cond_wait()` enquanto o sinal da sua direção estiver vermelho; `traffic_signal_loop()` alterna as fases (Norte/Sul verde ↔ Leste/Oeste verde) a cada ~2 segundos e dá broadcast a cada mudança de estado.

6. Ausência de Espera Ocupada

Nenhum carro parado aguardando o próximo tick ou um sinal verde consome CPU em loop. Os dois pontos de bloqueio de um veículo usam `pthread_cond_wait()`:

- `clock_wait_tick()` — o veículo dorme até o próximo tick do relógio global.
- `traffic_wait_green()` — o veículo dorme até que o semáforo do cruzamento libere sua direção.

A própria thread do semáforo (`traffic_signal_loop`) usa `usleep(100000)` para temporizar as fases do ciclo, em vez de um laço que testa o tempo continuamente. Dentro de cada fase, ela também faz pequenas pausas de 100 ms para verificar se há um pedido de prioridade de ambulância pendente — esse é um polling de baixa frequência com sleep entre as checagens, e não um busy-wait, já que a CPU fica livre durante cada intervalo de 100 ms.

7. Tratamento da Ambulância

A ambulância é um veículo do tipo `TYPE_AMBULANCE`, sempre com velocidade `SPEED_FAST`. Ao se aproximar de um cruzamento, o veículo chama `traffic_request_priority()`, que marca `priority_requested = 1` e força o estado de todas as direções do semáforo daquele cruzamento — verde apenas para a direção da ambulância — além de imprimir um aviso no terminal:

```
printf("\n>>> [PRIORIDADE] Ambulancia solicitou prioridade no cruzamento (%d, %d) indo para %s!\n", ...);
```

Enquanto `priority_requested` estiver ativo, `traffic_signal_loop()` interrompe o ciclo normal de fases e mantém o sinal fixo na direção solicitada. A prioridade é liberada automaticamente quando a ambulância deixa a célula do cruzamento (`cell_release()` detecta `occupant_symbol == 'A'` e zera `priority_requested`).

Como a entrada da ambulância na célula do cruzamento ainda passa por `cell_try_occupy()` e por `traffic_wait_green()` como qualquer outro veículo, a prioridade não quebra a regra de exclusão mútua: ela apenas garante que o sinal fique verde na direção certa mais rapidamente, sem permitir que dois veículos ocupem a mesma célula.

8. Estratégia contra Deadlocks

A estratégia adotada para evitar deadlocks é evitar completamente a espera bloqueante por células — ou seja, remover a condição de "hold-and-wait" para o recurso mais disputado do sistema (as células da pista):

- `cell_try_occupy()` é não bloqueante: o veículo tenta ocupar a próxima célula uma única vez. Se ela estiver ocupada, a função retorna 0 imediatamente e o veículo simplesmente não avança naquele tick, tentando novamente no tick seguinte.
- Nenhuma thread mantém o mutex de uma célula enquanto espera por outro recurso: o mutex de cada célula é mantido apenas durante a checagem/escrita de `occupant_id`, dentro de

`cell_try_occupy()/cell_release()`, e é liberado antes de qualquer bloqueio (como o `cond_wait` de `traffic_wait_green()` ou `clock_wait_tick()`).

- Como resultado, não existe cadeia de espera circular entre mutexes de célula: no pior caso, dois veículos tentando trocar de posição simplesmente falham em ocupar a célula um do outro e permanecem parados até o próximo tick, sem nunca ficar bloqueados segurando um recurso.

[CUIDADOS COM O MAPA] A equipe, durante os testes, notou que os veículos poderiam acabar se concentrando em um lado só do mapa, o que poderia gerar um engarrafamento entre um semáforo e outro e, conseqüentemente, um deadlock. Para isso, a estrutura do mapa foi modificada e foi alterada a lógica de “se a próxima célula for parede, vire à direita” inicialmente implementada.

9. Visualização

A visualização é feita em terminal/ASCII por `render_frame()`, chamada a cada tick pelo loop principal de `main.c`. A função limpa a tela com o escape ANSI `\033[2J\033[H` e redesenha o mapa inteiro, célula por célula, através de `cell_char()`, que prioriza mostrar o veículo ocupando a célula (`occupant_symbol`) sobre o tipo da via.

A legenda dos símbolos é impressa uma única vez, na primeira renderização, para não gerar saída redundante a cada tick.

10. Decisões de Implementação e Trade-offs

Esta seção reúne decisões de modelagem que vão além do mínimo pedido no enunciado e os trade-offs considerados pela equipe.

[RANDER_H] O sistema de geração de veículos foi, inicialmente, utilizando o `rand_r_h`, que não se mostrou interessante para usuários de Windows e teve que ser chamado por outros métodos “thread-safe”, conforme descrito no README.

[MAPA EM TXT] A malha do mapa está representada por um arquivo `.txt` pela facilidade de uso dos caracteres de texto e de suas alterações quando necessárias para representar entidades do sistema.

11. Aprendizados da Equipe

[DAS DEPENDÊNCIAS] Cada integrante, desde a primeira reunião inicial de planejamento, esteve ciente das dependências do trabalho, desde o que deve fazer ao que deve esperar que seja feito e todos agiram de acordo com os prazos estabelecidos pela equipe.

[DAS DIFICULDADES TÉCNICAS] Não havia no time aluno confortável com a sintaxe da linguagem C ou de profundo conhecimento na mesma. Então, a equipe decidiu direcionar bastante o foco em manter comentários nas linhas de código, nos padrões de commit e sempre manter revisores de códigos nos Pull Requests para o repositório, a fim de que todos, a todo momento, utilizassem da oportunidade para tirar dúvidas e ser cuidadosamente avaliado.

[DA INEXPERIÊNCIA COM INTERFACES] A equipe passou por um grande debate teórico sobre como sincronizar o estado lógico do sistema (Threads concorrentes controlando veículos e semáforos) com a sua respectiva representação puramente visual em texto ASCII.

[DA INEXPERIÊNCIA COM A LINGUAGEM C] O sistema de geração de veículos inicialmente utilizava uma biblioteca/mecanismo chamado `rander_h`, mas este apresentou problemas de compatibilidade e não se mostrou interessante ou funcional para os membros que utilizavam o sistema operacional Windows. A falta de um exemplo multiplataforma funcional exigiu a substituição dessa lógica por outros métodos de renderização que fossem estritamente thread-safe e portáteis, representados no README do projeto, onde contém instruções para usuários Windows driblarem esse impasse sem que seja necessário alterar o código fonte.

12. Compilação e Execução

O projeto é compilado com gcc e a flag `-pthread`, via Makefile:

```
make          # compila o binário em bin/simulador
make run      # compila (se necessário) e executa a simulação
make clean    # remove objetos e binário gerados
```

O binário também aceita argumentos opcionais: `./bin/simulador [caminho_do_mapa] [ticks_da_simulacao]`. Sem argumentos, usa `assets/map.txt` e um número de ticks padrão.

13. Checklist de Atendimento aos Requisitos

Autoavaliação da equipe em relação aos critérios do enunciado, com base no estado atual do código no repositório:

Requisito	Status	Observação
10 a 20 veículos como threads simultâneas	Atendido	MAX_VEHICLE está definido como 20 e main.c lança NUM_TOTAL_VEHICLES = MAX_VEHICLE. Falta validar em execução real quantas threads chegam a rodar simultaneamente sem engarrafar todos os spawns.
Pelo menos 1 ambulância com prioridade	Atendido	TYPE_AMBULANCE existe; traffic_request_priority() força verde na direção da ambulância assim que ela se aproxima de um cruzamento.
8+ cruzamentos com semáforos	Atendido	assets/map.txt possui 9 células '+' e sync_init() cria um TrafficSignal para cada uma.
Pelo menos 1 via de mão única	Atendido	map.txt usa '^', 'v', '>', '<' fora dos cruzamentos; vias de mão dupla são modeladas como duas faixas de mão única lado a lado.
Nenhum busy-wait nos bloqueios de carro	Atendido	clock_wait_tick() e traffic_wait_green() usam pthread_cond_wait().

Requisito	Status	Observação
Nenhuma race condition na ocupação de células	Atendido	Toda leitura/escrita de occupant_id passa por cell_try_occupy()/cell_release(), protegidas pelo mutex da célula.
Estratégia anti-deadlock definida	Atendido (ver seção 8)	Ocupação de célula é não bloqueante (trylock); nenhuma thread mantém um mutex de célula enquanto espera outro recurso.
Visualização em terminal/ASCII a cada tick	Atendido	render_frame() limpa a tela com \033[2J\033[H e redesenha o mapa e a legenda a cada chamada no loop principal do main.c.
README com instruções de compilação/execução	Atendido	README.md descreve o projeto; detalha make / make run / make clean e os argumentos de main.

14. Conclusão

Apesar das dificuldades técnicas - muitas que fogem do conteúdo programático, como a situação do rande_h, a dificuldade com Windows x Linux, discussões sobre virtualização ou containers do projeto e gerenciamento de tempo - a equipe foi capaz de desenvolver um trabalho satisfatório que atenda aos requisitos exigidos pelo projeto prático mas também às exigências de qualidade dos membros participantes. É de consenso geral da equipe que seria muito interessante voltar nesse exercício futuramente quando houver por parte dos integrantes maior bagagem técnica para tomada de decisões e estratégias.

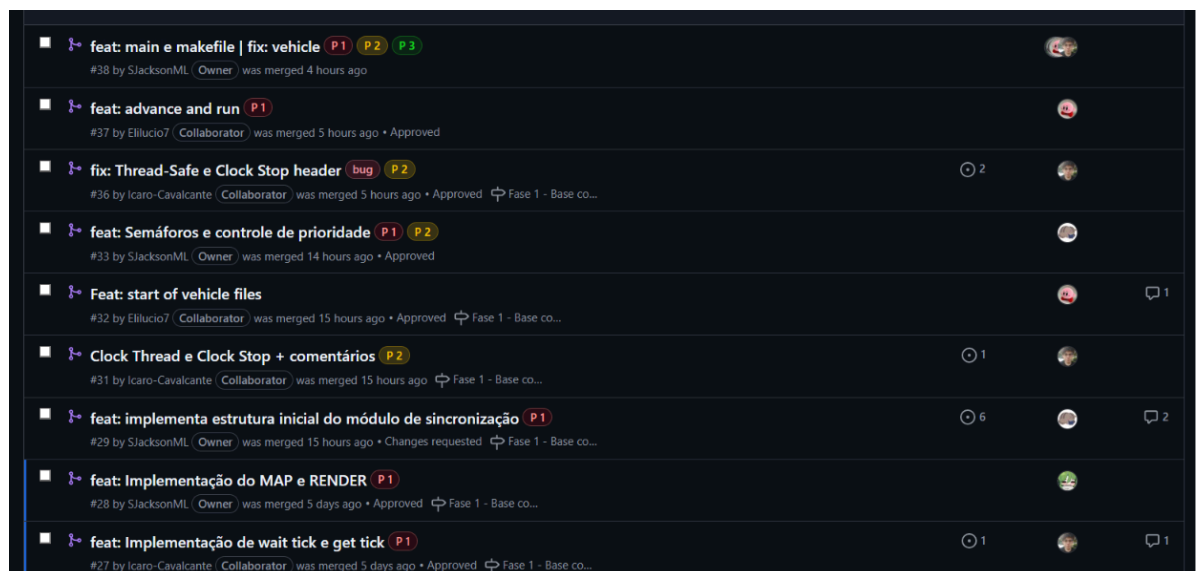
Sobre o trabalho, foi interessante trabalhar com uma interface sem trabalhar com frontend. Grande parte da discussão teórica se deu justamente sobre como sincronizar os processos do sistema com a representação visual em ASCII.

15. Repositório

Link do repositório GitHub: <https://github.com/SJacksonML/SO-Gerenciamento-de-Trafego.git>

O histórico de commits, bem como issues e milestones, identifica a participação de cada integrante, conforme exigido no enunciado.

Figura — representação do histórico de commits com padrão de títulos, marcadores de Prioridade por tags, comentários de outros membros e issues associados.



feat: main e makefile fix: vehicle	P1 P2 P3	#38 by SJacksonML (Owner) was merged 4 hours ago
feat: advance and run	P1	#37 by Eliucio7 (Collaborator) was merged 5 hours ago • Approved
fix: Thread-Safe e Clock Stop header	bug P2	#36 by Icaro-Cavalcante (Collaborator) was merged 5 hours ago • Approved ↗ Fase 1 - Base co...
feat: Semáforos e controle de prioridade	P1 P2	#33 by SJacksonML (Owner) was merged 14 hours ago • Approved
Feat: start of vehicle files		#32 by Eliucio7 (Collaborator) was merged 15 hours ago • Approved ↗ Fase 1 - Base co...
Clock Thread e Clock Stop + comentários	P2	#31 by Icaro-Cavalcante (Collaborator) was merged 15 hours ago • Approved ↗ Fase 1 - Base co...
feat: implementa estrutura inicial do módulo de sincronização	P1	#29 by SJacksonML (Owner) was merged 15 hours ago • Changes requested ↗ Fase 1 - Base co...
feat: Implementação do MAP e RENDER	P1	#28 by SJacksonML (Owner) was merged 5 days ago • Approved ↗ Fase 1 - Base co...
feat: Implementação de wait tick e get tick	P1	#27 by Icaro-Cavalcante (Collaborator) was merged 5 days ago • Approved ↗ Fase 1 - Base co...